

14.3 算术和关系运算符

通常情况下，我们把算术和关系运算符定义成非成员函数以允许对左侧或右侧的运算对象进行转换（参见 14.1 节，第 492 页）。因为这些运算符一般不需要改变运算对象的状态，所以形参都是常量的引用。

算术运算符通常会计算它的两个运算对象并得到一个新值，这个值有别于任意一个运算对象，常常位于一个局部变量之内，操作完成后返回该局部变量的副本作为其结果。如果类定义了算术运算符，则它一般也会定义一个对应的复合赋值运算符。此时，最有效的方式是使用复合赋值来定义算术运算符：

```
// 假设两个对象指向同一本书
Sales_data
operator+(const Sales_data &lhs, const Sales_data &rhs)
{
    Sales_data sum = lhs;           // 把 lhs 的数据成员拷贝给 sum
    sum += rhs;                     // 将 rhs 加到 sum 中
    return sum;
}
```

这个定义与原来的 add 函数（参见 7.1.3 节，第 234 页）是完全等价的。我们把 lhs 拷贝给局部变量 sum，然后使用 Sales_data 的复合赋值运算符（将在第 500 页定义）将 rhs 的值加到 sum 中，最后函数返回 sum 的副本。



如果类同时定义了算术运算符和相关的复合赋值运算符，则通常情况下应该使用复合赋值来实现算术运算符。

561

14.3 节练习

练习 14.13: 你认为 Sales_data 类还应该支持哪些其他算术运算符（参见表 4.1，第 124 页）？如果有的话，请给出它们的定义。

练习 14.14: 你觉得为什么调用 operator+= 来定义 operator+ 比其他方法更有效？

练习 14.15: 你在 7.5.1 节的练习 7.40（第 261 页）中曾经选择并编写了一个类，你认为它应该含有其他算术运算符吗？如果是，请实现它们；如果不是，解释原因。

14.3.1 相等运算符



通常情况下，C++ 中的类通过定义相等运算符来检验两个对象是否相等。也就是说，它们会比较对象的每一个数据成员，只有当所有对应的成员都相等时才认为两个对象相等。依据这一思想，我们的 Sales_data 类的相等运算符不但应该比较 bookNo，还应该比较具体的销售数据：

```
bool operator==(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() == rhs.isbn() &&
           lhs.units_sold == rhs.units_sold &&
           lhs.revenue == rhs.revenue;
}

bool operator!=(const Sales_data &lhs, const Sales_data &rhs)
```

```
{
    return !(lhs == rhs);
}
```

就上面这些函数的定义本身而言，它们似乎比较简单，也没什么价值，对于我们来说重要的是从这些函数中体现出来的设计准则：

- 如果一个类含有判断两个对象是否相等的操作，则它显然应该把函数定义成 `operator==` 而非一个普通的命名函数：因为用户肯定希望能使用 `==` 比较对象，所以提供了 `==` 就意味着用户无须再费时费力地学习并记忆一个全新的函数名字。此外，类定义了 `==` 运算符之后也更容易使用标准库容器和算法。
- 如果类定义了 `operator==`，则该运算符应该能判断一组给定的对象中是否含有重复数据。
- 通常情况下，相等运算符应该具有传递性，换句话说，如果 `a==b` 和 `b==c` 都为真，则 `a==c` 也应该为真。
- 如果类定义了 `operator==`，则这个类也应该定义 `operator!=`。对于用户来说，当他们能使用 `==` 时肯定也希望使用 `!=`，反之亦然。
- 相等运算符和不相等运算符中的一个应该把工作委托给另外一个，这意味着其中一个运算符应该负责实际比较对象的工作，而另一个运算符则只是调用那个真正工作的运算符。

562

Best
Practices

如果某个类在逻辑上有相等性的含义，则该类应该定义 `operator==`，这样做可以使得用户更容易使用标准库算法来处理这个类。

14.3.1 节练习

练习 14.16: 为你的 `StrBlob` 类（参见 12.1.1 节，第 405 页）、`StrBlobPtr` 类（参见 12.1.6 节，第 421 页）、`StrVec` 类（参见 13.5 节，第 465 页）和 `String` 类（参见 13.5 节，第 470 页）分别定义相等运算符和不相等运算符。

练习 14.17: 你在 7.5.1 节的练习 7.40（第 261 页）中曾经选择并编写了一个类，你认为它应该含有相等运算符吗？如果是，请实现它；如果不是，解释原因。



14.3.2 关系运算符

定义了相等运算符的类也常常（但不总是）包含关系运算符。特别是，因为关联容器和一些算法要用到小于运算符，所以定义 `operator<` 会比较有用。

通常情况下关系运算符应该

1. 定义顺序关系，令其与关联容器中对关键字的要求一致（参见 11.2.2 节，第 378 页）；并且
2. 如果类同时也含有 `==` 运算符的话，则定义一种关系令其与 `==` 保持一致。特别是，如果两个对象是 `!=` 的，那么一个对象应该 `<` 另外一个。



尽管我们可能会认为 `Sales_data` 类应该支持关系运算符，但事实证明并非如此，其中的缘由比较微妙，值得读者深思。

一开始我们可能会认为应该像 `compareIsbn`（参见 11.2.2 节，第 379 页）那样定义 `<`，该函数通过比较 ISBN 来实现对两个对象的比较。然而，尽管 `compareIsbn` 提供的

顺序关系符合要求 1, 但是函数得到的结果显然与我们定义的 `==` 不一致, 因此它不满足要求 2。

对于 `Sales_data` 的 `==` 运算符来说, 如果两笔交易的 `revenue` 和 `units_sold` 成员不同, 那么即使它们的 `ISBN` 相同也无济于事, 它们仍然是不相等的。如果我们定义的 `<` 运算符仅仅比较 `ISBN` 成员, 那么将发生这样的情况: 两个 `ISBN` 相同但 `revenue` 和 `units_sold` 不同的对象经比较是不相等的, 但是其中的任何一个都不比另一个小。然而实际情况是, 如果我们有两个对象并且哪个都不比另一个小, 则从道理上来讲这两个对象应该是相等的。

563

基于上述分析我们也许会认为, 只要让 `operator<` 依次比较每个数据元素就能解决问题了, 比方说让 `operator<` 先比较 `isbn`, 相等的话继续比较 `units_sold`, 还相等再继续比较 `revenue`。

然而, 这样的排序没有任何必要。根据将来使用 `Sales_data` 类的实际需要, 我们可能会希望先比较 `units_sold`, 也可能希望先比较 `revenue`。有的时候, 我们希望 `units_sold` 少的对象“小于”`units_sold` 多的对象; 另一些时候, 则可能希望 `revenue` 少的对象“小于”`revenue` 多的对象。

因此对于 `Sales_data` 类来说, 不存在一种逻辑可靠的 `<` 定义, 这个类不定义 `<` 运算符也许更好。

Best
Practices

如果存在唯一一种逻辑可靠的 `<` 定义, 则应该考虑为这个类定义 `<` 运算符。如果类同时还包含 `==`, 则当且仅当 `<` 的定义和 `==` 产生的结果一致时才定义 `<` 运算符。

14.3.2 节练习

练习 14.18: 为你的 `StrBlob` 类、`StrBlobPtr` 类、`StrVec` 类和 `String` 类定义关系运算符。

练习 14.19: 你在 7.5.1 节的练习 7.40 (第 261 页) 中曾经选择并编写了一个类, 你认为它应该含有关系运算符吗? 如果是, 请实现它; 如果不是, 解释原因。

14.4 赋值运算符

之前已经介绍过拷贝赋值和移动赋值运算符 (参见 13.1.2 节, 第 443 页和 13.6.2 节, 第 474 页), 它们可以把类的一个对象赋值给该类的另一个对象。此外, 类还可以定义其他赋值运算符以使用别的类型作为右侧运算对象。

举个例子, 在拷贝赋值和移动赋值运算符之外, 标准库 `vector` 类还定义了第三种赋值运算符, 该运算符接受花括号内的元素列表作为参数 (参见 9.2.5 节, 第 302 页)。我们能以如下的形式使用该运算符:

```
vector<string> v;
v = {"a", "an", "the"};
```

同样, 也可以把这个运算符添加到 `StrVec` 类中 (参见 13.5 节, 第 465 页):

```
class StrVec {
public:
    StrVec &operator=(std::initializer_list<std::string>);
    // 其他成员与 13.5 节 (第 465 页) 一致
};
```